

Final Report

COMPENG 2DX3

Claresta Cheung

400582489

April 7th, 2026

As a future member of the engineering profession, the student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is our own and adheres to the Academic Integrity Policy of McMaster University and the Code of Conduct of the Professional Engineers of Ontario.

Table of Contents

Table of Contents.....	1
Device Overview.....	2
Features.....	2
General description.....	2
Block diagram (data flow graph).....	4
Device Characteristics (user’s technical ‘need to know’).....	4
Distance Measurement: Acquisition.....	6
Detailed description.....	6
Measurements.....	6
Visualization.....	8
Application Example with Expected Output and Brief Users Guide.....	9
Application Example.....	9
Limitations.....	10
Circuit Schematic and Programming Logic Flowchart.....	11
Circuit Schematic.....	11
Programming Logic Flowchart.....	12

Device Overview

Features

This device features multiple components that work together to produce a 3D scan of the surrounding environment of an intelligent microsystem. The hardware components involved in this design include: MSP432E401Y SimpleLink™ Ethernet Microcontroller, VMA401 Stepper Motor, TI ULN2003 Driver Board, and VL53L1X ToF Sensor. The software components involved in this design include: Keil and Python.

The MSP432E401Y SimpleLink™ Ethernet Microcontroller features a 120 MHz Core (CPU Bus Speed) with Floating-Point Unit (FPU). This uses a 25 MHz External Precision Crystal (MOSC). If using PIOSC, a 16 MHz system clock is used. The peripheral bus speed depends on what interface is used (I²C, UART, etc).

The operating voltage used is 3.3 V for the VL53L1X ToF Sensor and 5 V for the VMA401 Stepper Motor connected to the microcontroller via a TI ULN2003 Driver Board. 3.3 V is used for the ToF sensor to maintain logic-level compatibility with the I2C peripheral. 5 V is used for the Stepper Motor to provide the necessary current and sufficient torque for a 360° rotation.

According to the provided datasheets for the components, this device is relatively low-cost. Online, the cost of all components totalled together is around \$78 to \$89 depending on where the components are purchased.

The serial communication used in this device uses I²C protocol (synchronous) for communication between the ToF sensor and the microcontroller. The microcontroller then uses UART communication (asynchronous) with a baud rate of 115 200 bits per second with the PC.

The programming language in the receiver side is chosen by the designer based on familiarity with user-interface - Python. Data is transmitted via UART, and captured on PC using pyserial. Open3D is then used to generate a 3D scan plot.

The MSP432E401Y has 1024 KB of flash memory and 256 KB of SRAM.

General description

Signal acquisition is accomplished via ToF sensor. This sensor has an emitter which emits a 940 nm invisible laser. This sensor has a receiver which receives reflected light after the laser reflects off of an object/surface. A distance measurement value is then returned.

During preprocessing, the digital signal is processed on-chip. The actual distance is then calculated in millimeters.

During the Analog to Digital Conversion (ADC), I²C protocol is used to transmit signal from the ToF sensor to the microcontroller.

Serial communication/communication with PC methods is done via UART (asynchronous) communication. The configured Baud rate is 115 200 baud. A USB-B cord is used as a serial COM port to send data to the PC. A python script using pyserial captures points and plots on a graph using Open3D.

Visualization is first achieved with the ToF distance measurement. The equation (*Equation 1*) used is as follows:

$$\text{Measured distance} = \left(\frac{\text{photon travel time}}{2} \right) * (\text{speed of light})$$

Equation 1: ToF Distance Measurement

This equation has travel time divided by 2 because the laser emitted travels twice the distance: once towards the surface reflecting the laser, another when going back to the ToF sensor.

Given distance d , x and y coordinates can be calculated using the following equations (*Equations 2*):

$$x = d * \cos(\theta)$$

$$y = d * \sin(\theta)$$

Equations 2: x and y coordinates

z is the distance physically moved in the direction of the cross product of x and y . Although x coordinate in manual is assigned to be the distance between scans as the prototype moves forward to proceed with each individual scan, coordinate system is arbitrarily decided for ease of use and debugging - hence why z is distance moved forward.

This logic uses high school trigonometry of a right angled triangle, where d is the hypotenuse in the context of Pythagorean Theorem.

Block diagram (data flow graph)

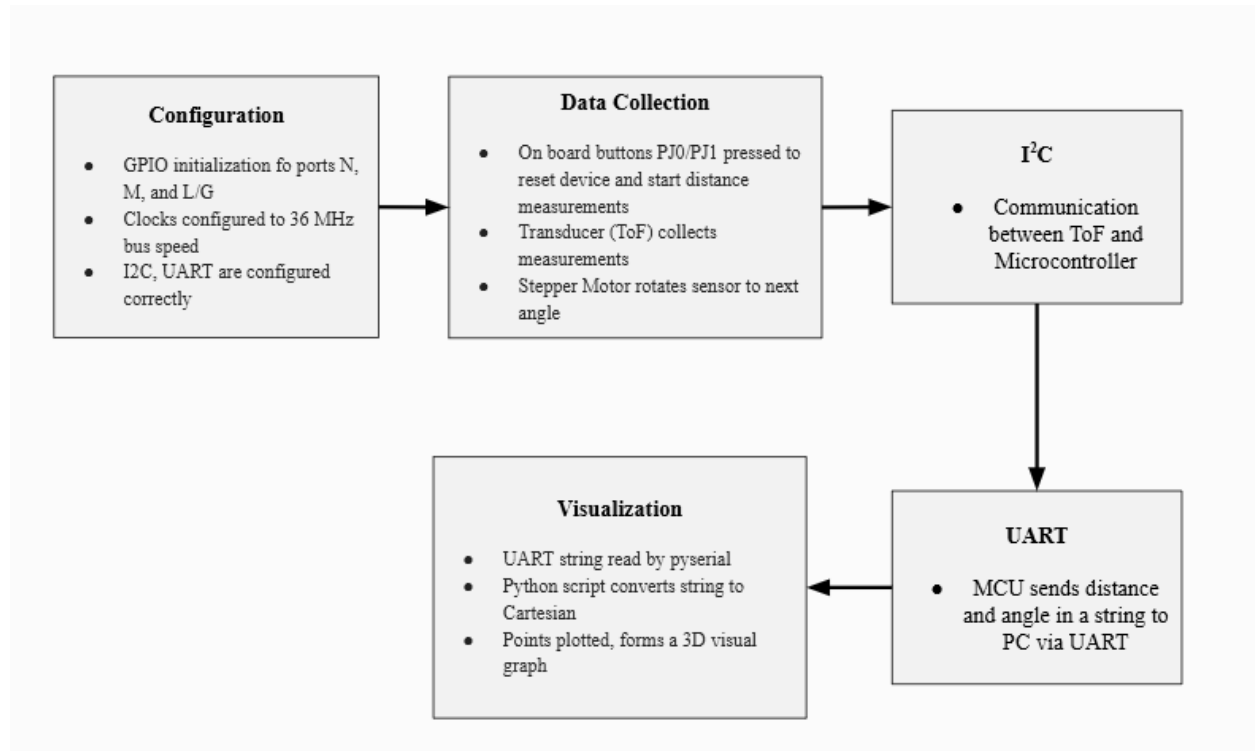


Figure 1: Block diagram

Device Characteristics (user's technical 'need to know')

Based on the following Student Number: 400582489, the assigned pin numbers and bus speed are listed in *Table 1*. The serial port/communication speed used is 115 200 bits/second. The pins used and wiring specifications are listed in *Table 2* for the microcontroller, *Table 3* for the stepper motor, and *Table 4* for the ToF sensor.

Table 1: Device Characteristics

Characteristic	Student ID Digit	Assigned Value
Bus Speed	LSB: 9	36 MHz
Measurement Status LED	2nd LSB: H	PN1
UART LED	2nd LSB: H	PN0

Additional Status LED	2nd LSB: H	PF4
-----------------------	------------	-----

Table 2: Microcontroller Characteristics

Push buttons	PJ0, reset
Serial Port	COM5
Baud Rate	115200

Table 3: Stepper Motor Characteristics

In 1	PM0
In 2	PM1
In 3	PM2
In 4	PM3
V ⁺	5V
V ⁻	GND

Table 4: ToF Sensor Characteristics

V _{in}	3.3V
GND	GND
SDA	PB3
SCL	PB2

Distance Measurement: Acquisition

The core component of Acquisition of data is achieved through the ToF sensor. A ToF sensor measures how long it takes for emitted pulses of light to reach the nearest object and be reflected to the detector. The VL53L1X ToF emits pulses of infrared laser light using an emitter. Then, the signal reflects from the nearest object (e.g. wall). This reflection is then collected by the ToF receiver. Finally, the time delay between the emitted and received periodic signal is measured.

As mentioned prior under *General Description*, the distance measurement is achieved through mathematical calculations by using *Equation 1*. The VL53L1Z ToF sensor undergoes transduction, conditioning, and Analog to Digital Conversion (ADC). It then returns measurements in mm (millimeters) and data is transmitted to the microcontroller via I2C communication.

The number of measurements taken was 64, as the stepper motor was configured to rotate and stop at every 5.625 deg. This number was arbitrarily chosen to capture more data points, ensuring a smoother and accurate display of measurements taken. To ensure all 64 measurements have been recorded properly, checking the number of data measurements sent from the microcontroller to PC is done. This can be seen through a .xyz file created via Python script generated, or through other softwares for troubleshooting (Realterm or PuTTY).

The Acquisition of Data points must first be invoked by the user through flashing the C code from Keil to the microcontroller. The reset button is pressed to ensure the system starts fresh. When the user is ready to start recording data points, PJ0 is pressed. PJ0 is SW1, which was previously initialized in the Keil code.

Detailed description

Measurements

A right angled triangle is used for visualization purposes (*Figure 2*) in describing how distance and angle measurement is converted to the xyz plane. These calculations/conversions are done in the Python code.

By using equations from *Equations 2*, x and y coordinates can be found. The equations are re-listed below:

$$x = d * \cos(\theta)$$

$$y = d * \sin(\theta)$$

It has been stated under *General Description* that the z-coordinate chosen is the displacement the user chooses to move the sensor forward as the measurements are obtained.

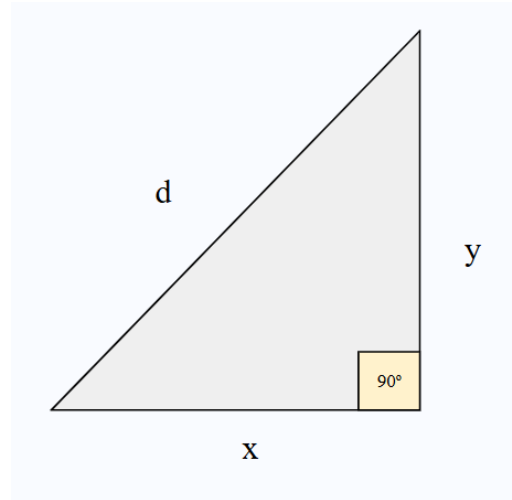


Figure 2: Right angled Triangle

The distance measurement has already been defined under *General Description* in *Equation 1*, which is re-listed below:

$$\text{Measured distance} = \left(\frac{\text{photon travel time}}{2} \right) * (\text{speed of light})$$

If the ToF sensor measures a photon travel time of 8.34 ns, the calculations are as such:

$$\begin{aligned} \text{Measured distance} &= \left(\frac{8.34 \times 10^{-9}}{2} \right) * 3 \times 10^8 \\ &= 4.17 \times 10^{-9} * 3 \times 10^8 \\ &= 1.251 \end{aligned}$$

This example distance calculation is 1.251 meters, which translates to 1251 mm.

Displacement is implemented by physical movement of the scanner by the user forward by a fixed value - say 100 mm - after a full rotation of the stepper motor. This displacement is referred to in the coordinate system as the z axis, as defined earlier. As well, the Python script tracks this displacement by a variable *Scan Index*. After each 'slice', this *Scan Index* is incremented by 100 if the fixed value is 100 mm.

Visualization

The visualization software chosen to be implemented is Python. Data transmission and 3D graphing is achieved through programming a Python script on Python and importing the following libraries: serial, math, numpy, open3d, and time.

Below are the explicit definitions of the libraries used:

Pyserial is used to manage the UART between the PC and microcontroller at a Baud rate of 115 200 baud. Open3D is a modern library used to render 3D geometry and points, generating a user-friendly and interactive graph for easy visualization. The programmer can also choose to implement colors into the Python code to achieve an aesthetic graph for visualization. Numpy provides high-performance multidimensional array objects - used to store xyz coordinates. Math supplies the trigonometric functions required for converting distance into xyz coordinates. Such functions include sine and cosine. Time is used to manage data-polling, as polling is chosen by the programmer to use over interrupts. Within the Python script, a few things are defined, such as: port used, baud rate, points per slice, step distance, and max time (optional, but used for safety timeout).

Figure 3 shows an example visualization comparison of the hallway to be scanned and the final scan completed with the code.

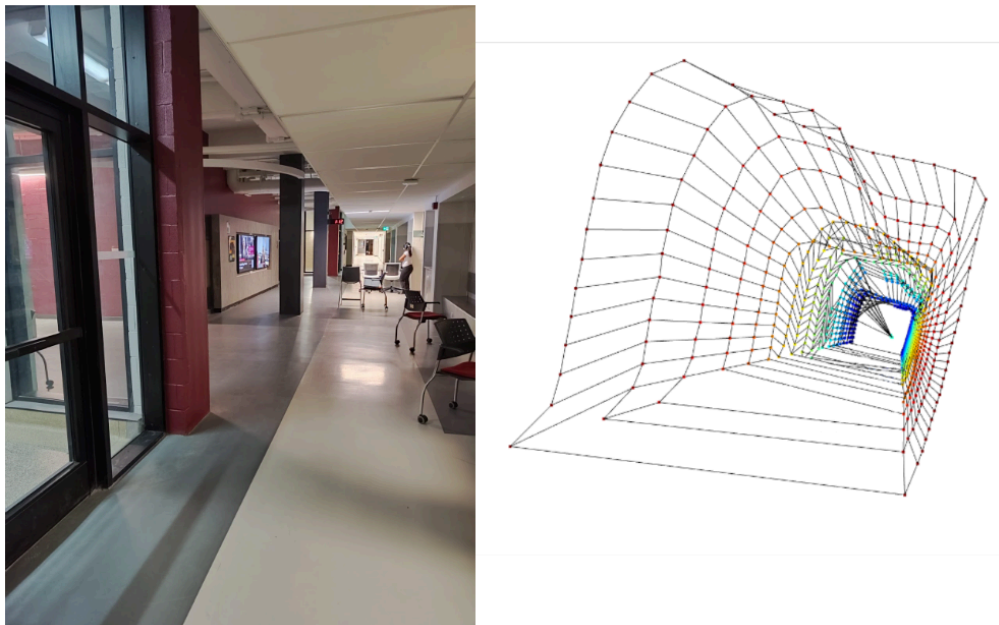


Figure 3: Hallway scan

Application Example with Expected Output and Brief Users Guide

Application Example

This configuration is applicable in real-life scenarios and devices, such as: a robot vacuum. A robot vacuum must implement motors to spin and scanners to map out a room to clean. This is crucial in intelligent microsystems as it can improve the quality of life of humans - as some may have difficulty maintaining the proper hygienic routine within their area of residence due to health problems, such as: back problems, aging, etc. It is also used by users to improve efficiency when one is too busy to clean up accumulating dust bunnies. This personalization of cleaning the whole mapped-out room becomes highly desirable in promising clean and effective cleaning services powered by the minds of engineers.

Below are the steps to use/setup the device:

1. Given microcontroller has the correct hardware setup (using Device Characteristics table), plug microcontroller into computer via USB
2. Ensure correct COM port enabled for correct data receiving for pyserial
3. Correct COM port is found by pressing the Windows key, searching for 'Device Manager' and opening it up
4. Look under Ports (COM & LPT), and hit the arrow to drop down for more options
5. Identify the COM # of the selection labelled with either UART or USB
6. Open Python through platforms like VS code
7. Change the COM port # if needed (within the python script) to the one found in step 5
8. Open Keil project. Press Translate, Build, and Download near the top left of the Keil interface
9. Press the on board reset button on the microcontroller (the button on the same side as the USB-B port)
10. LED D2 should light up to signal device is ready for data collection
11. Position the physical setup to measure an area the user would like to measure
12. Run Python script
13. Press the on board PJ0 (SW1) when user is ready to measure
14. LED D4 should flash once as soon as distance measurement starts
15. LED D3 should flash continuously to indicate measurement status
16. Repeat step 13 until user has gathered enough desired slices
17. When finished, press Ctrl C via Python terminal
18. A 3D model of scanned area is displayed via Open3D
19. To view xyz coordinates, user can open file named hallway_scan.xyz

To reiterate from prior sections, x and y are used for vertical distance slices. z is used for displacement. These are the definitions of how xyz is defined. For further description of these coordinates and how they are obtained, refer to *General Description*.

Limitations

Single Precision Floating Point Unit (FPU) is one of the key features of the microcontroller. This is a limitation because it lacks hardware support for high, double precision math, but can handle basic decimals. Using trigonometric functions becomes a limitation because of the computational cost it brings. Mathematically, power series (from Calc 2) are used for these on-chip functions, which consumes a great number of CPU cycles.

Put together, when performing 64 coordinate calculations for each rotation at 36 MHz, this causes the possibility of missing correct timing for stepper motor rotation or I2C bus polling. This may cause significant timing and data loss issues. This is resolved by using external environments such as Python to calculate xyz coordinates after obtaining distance measurements.

Another limitation would be the wires getting tangled up while the motor is turning. The motor is observed to occasionally be unable to rotate the full 360° , causing slight deviations of the actual environment being scanned to the 3D graph produced.

The maximum standard serial communication rate that can be implemented with the PC is 115 200 Baud. This is verified using the AD3. By probing an arbitrary header pin (for example PA2 or PD0, must be initialized by modifying Keil code), which is toggled every time a UART byte is sent, the scope in Waveforms is measured via the AD3. The correct value can be verified by ensuring each bit period is approximately 8.68 μ s. Protocol and Logic analyzer can be opened on Waveforms for thorough inspection and verification.

The communication method used between the microcontroller and ToF modules is I2C communication. This form of communication uses a synchronized clock system, as SCL is connected via ToF to the microcontroller. The speed used between them is implemented at a Fast Mode - an operating clock frequency of 400 kbps. This rate ensures quick retrieval of data and prevents bottlenecks.

The stepper motor poses a limitation on speed as a delay is required between each motor rotation phase. Another element that poses a limitation on speed is the sensor. This element requires time to collect enough photons for accurate reading. Sending data as ASCII strings during UART transmission also limits speed. To verify and test this, I can decrease SysTick_Wait in the motor

code. As the code runs faster, the motor would soon vibrate instead of turning normally, proving the motor being a limitation on speed.

Circuit Schematic and Programming Logic Flowchart

Circuit Schematic

Below is the Circuit Schematic (*Figure 4*), connecting the Stepper Motor and ToF sensor to the microcontroller. No external push buttons were used, only PJ0 on-board push button was configured and used to provide ease of use for the user.

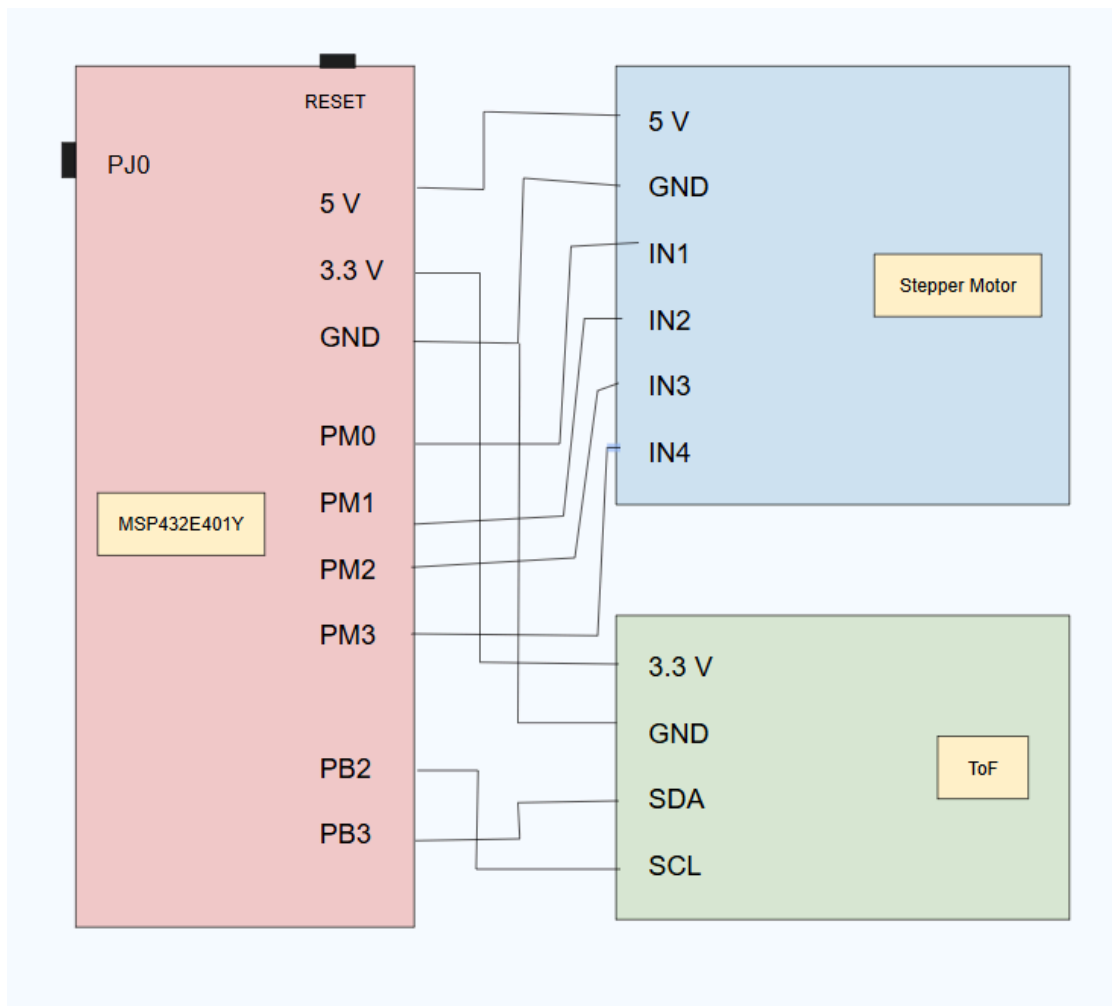
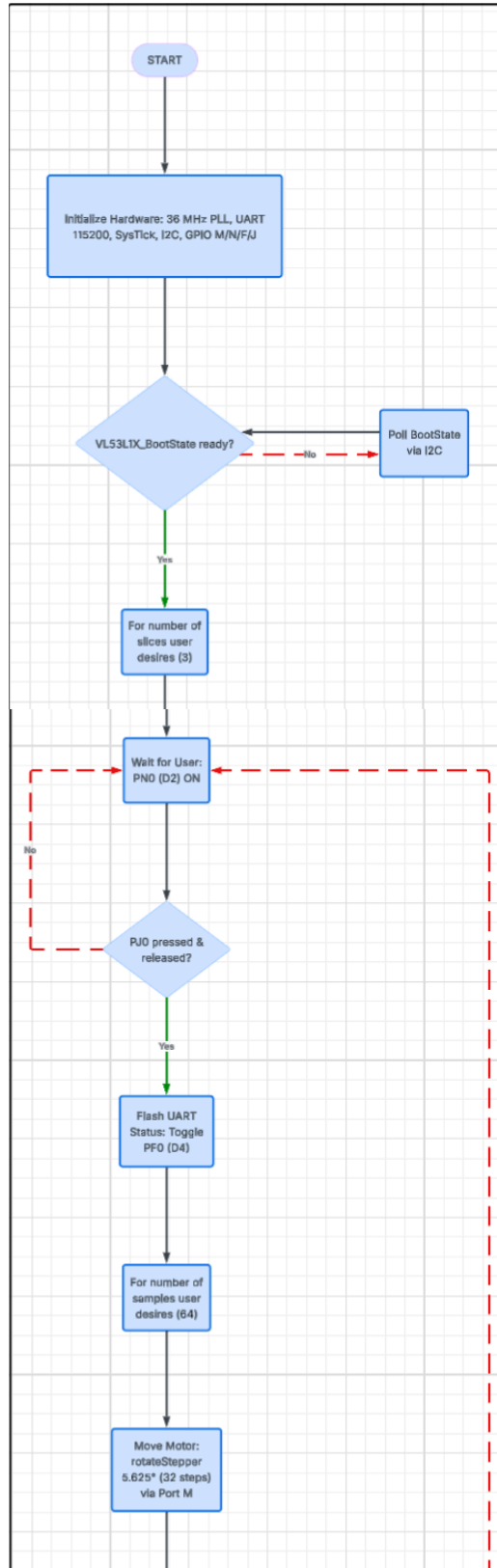


Figure 4: Circuit Schematic

Programming Logic Flowchart

Below are the Programming Logic Flowcharts for the microcontroller code on Keil (*Figure 5*) and the PC code on Python (*Figure 6*).



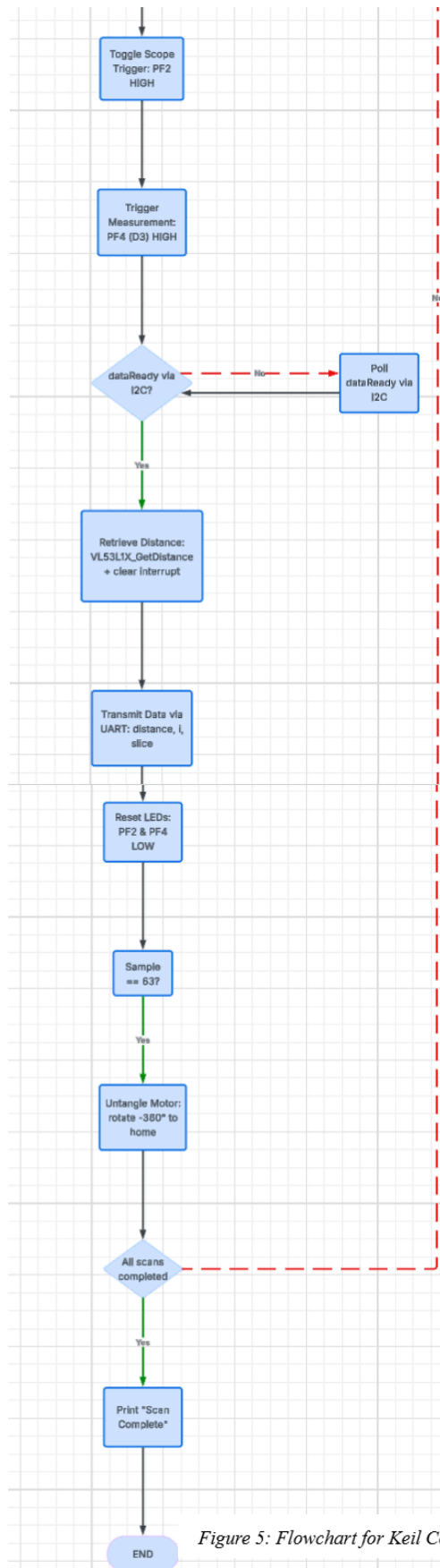


Figure 5: Flowchart for Keil Code



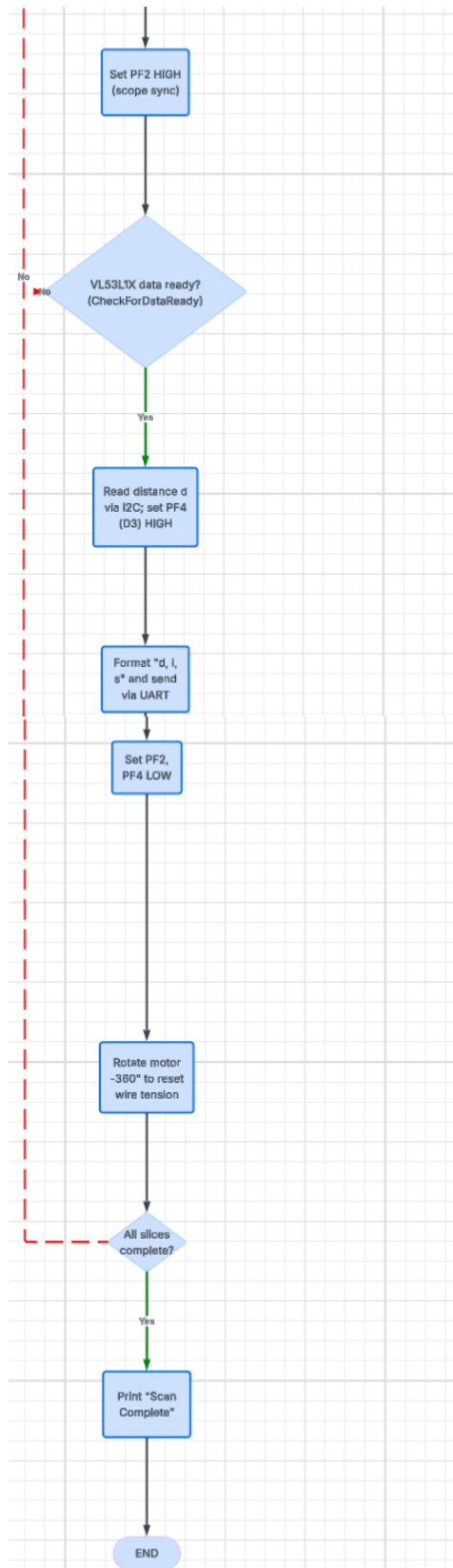


Figure 6: Flowchart for PC Code on Python